Sistema de gestão comercial e de inventário para uma loja de colecionáveis (Hobby Store)

Programação Orientada a Objetos

Nome: Maria Eduarda Menon **Turma:** TB

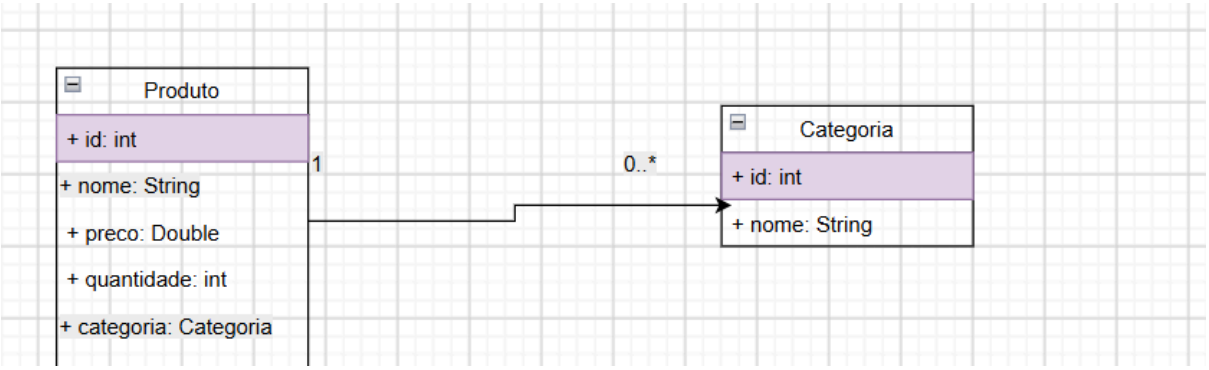
Dentro da nossa loja de hobbys, o controle de gestão foi construído com base em duas entidades: Categoria e Produto.

- Categoria: É a classificação geral a que os produtos pertencem (Ex: cartas; acessórios; jogos de tabuleiro) Cada categoria possui um identificador único (id) e um (nome)

- Produtos: É o artigo individual e específico que está à venda na loja, contendo o seu próprio nome, preço e stock (Ex: *Booster Box Pokémon*, *Proteções de Cartas (Sleeves)*, *Dado de 20 faces) Tem os dados do stock, como o preço unitário (preco), a quantidade disponível em armazém (quantidade) e o (nome)*

A ligação baseia-se na regra de que uma Categoria pode ter vários Produtos, mas um Produto pertence obrigatoriamente a uma única Categoria (1-N) um para muitos

Base de dados:



Requisitos:

1. Configuração do Projeto:

O projeto foi desenvolvido em Java utilizando o ambiente de desenvolvimento integrado (IDE) e as seguintes tecnologias e bibliotecas:

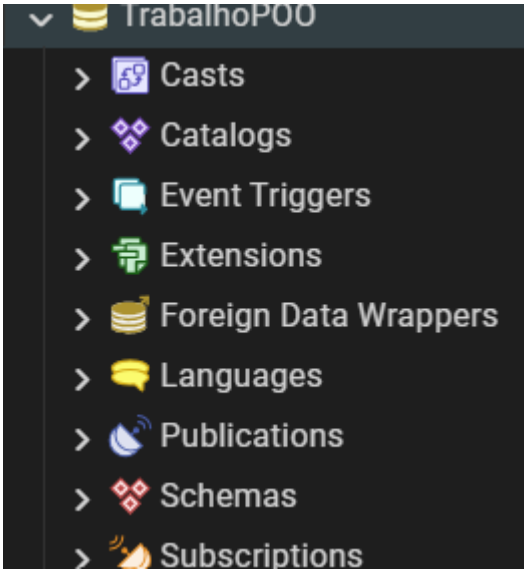
Linguagem: Java

Interface Gráfica: JavaFX para a construção dos componentes visuais

Gestão de Dependências: Maven

Persistência de Dados: Driver JDBC para a conexão com o PostgreSQ

2. Configuração do Banco de Dados:



Apagar tabelas antigas se existirem (para limpar a base de dados do zero)

```
DROP TABLE IF EXISTS item_venda CASCADE;
DROP TABLE IF EXISTS venda CASCADE;
DROP TABLE IF EXISTS produto CASCADE;
DROP TABLE IF EXISTS categoria CASCADE;
```

Criar a tabela de categorias

```
CREATE TABLE categoria (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);
```

	id [PK] integer	nome character varying (100)
1	1	Cartas
2	2	Acessórios
3	3	Jogos de Tabuleiro

Inserir as categorias principais com IDs fixos

```
INSERT INTO categoria (id, nome) VALUES (1, 'Cartas');
INSERT INTO categoria (id, nome) VALUES (2, 'Acessórios');
```

INSERT INTO categoria (id, nome) VALUES (3, 'Jogos de Tabuleiro');

Criar a tabela de produtos

```
CREATE TABLE produto (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL UNIQUE,
    preco NUMERIC(10, 2) NOT NULL,
    quantidade INT NOT NULL,
    categoria_id INT NOT NULL,
    CONSTRAINT fk_categoria FOREIGN KEY (categoria_id) REFERENCES
categoria(id)
);
```

(com UNIQUE no nome para evitar duplicatas)

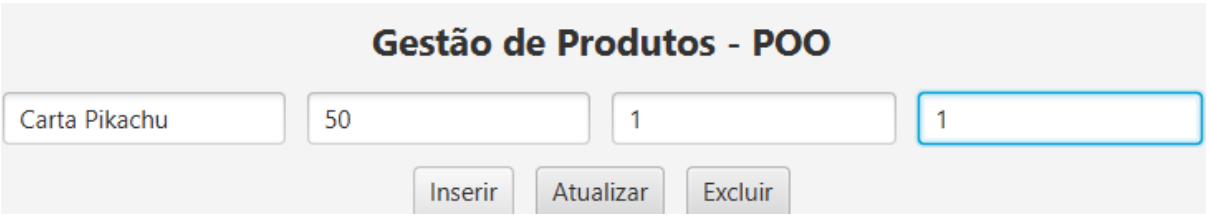
	id [PK] integer	nome character varying (100)	preco numeric (10,2)	quantidade integer	categoria_id integer
1	1	Booster Pokémon TCG	12.00	20	1
2	2	Booster Yu-Gi-Oh!	30.00	50	1
3	3	Booster Magic	25.00	40	1
4	4	Sleeves/Protetores de Cartas (Pacote c/ 1...	10.00	60	2
5	5	Kit de Dados RPG	45.00	30	2
6	6	Catan - Jogo de Tabuleiro	280.00	10	3
7	7	Dixit - Jogo de Tabuleiro	240.00	12	3

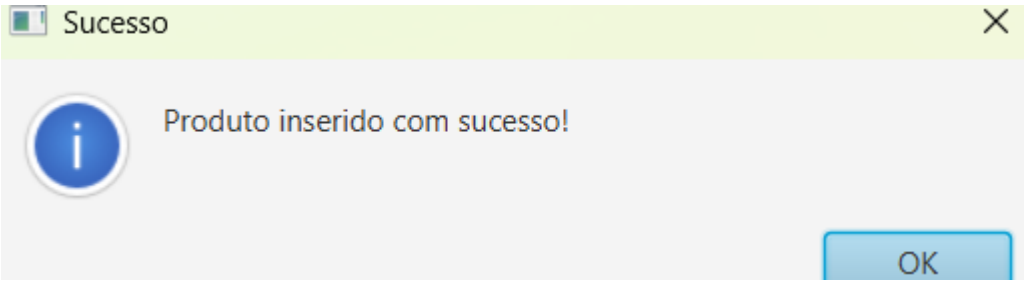
Nota: Inicialmente, a tabela de produtos exibia uma coluna com o **ID** sequencial de cadastro de cada item. No entanto, optei por removê-la da interface para deixar o design mais limpo nos dados comerciais (Nome, Preço, Quantidade e Categoria). O identificador técnico continua a existir internamente na base de dados para garantir o controlo correto dos registos.

3. Funcionalidades CRUD:



Inserir: Adiciona um novo registo de produto na base de dados

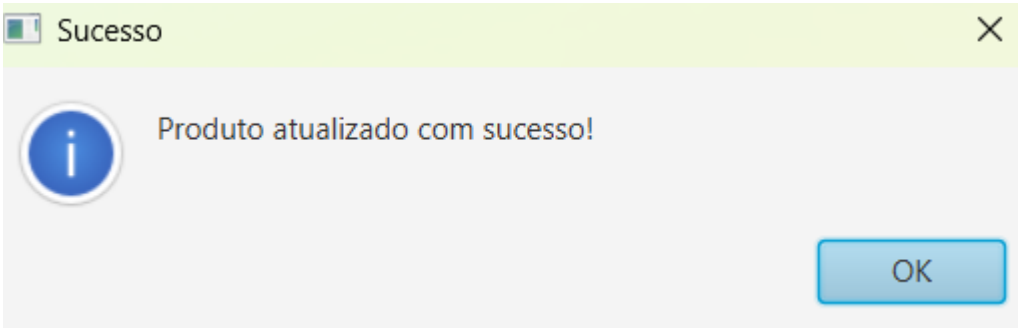






Atualizar: Modifica os dados de um produto que já existe e que foi previamente selecionado

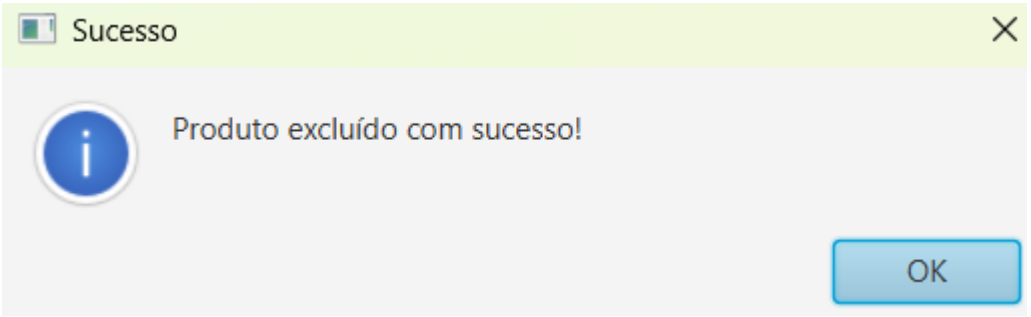






Excluir: Elimina permanentemente o produto selecionado







Consultar: A listagem e exibição dos produtos registados é feita automaticamente na tabela (TableView) através do carregamento dos dados vindos do PostgreSQL.

4. Padrão MVC:

A aplicação foi arquitetada utilizando o padrão de projeto *Model-View-Controller* (MVC) para garantir a separação de responsabilidades e a organização em camadas:

Model: Contém as entidades de negócio (Produto e Categoria) e a classe de acesso a dados (ProdutoDAO), responsável pelas operações SQL.

View: Composta pelo ficheiro FXML (produto_view.fxml), definindo a interface gráfica do utilizador em JavaFX.

Controller: A classe ProdutoController atua como intermediária, capturando os eventos da interface e acionando as regras de negócio e persistência na camada Model.

